

Risk Management

Sopify AI-Assisted SDLC — Risk Methodology, Register, Controls and Escalation

Document ID	RISK-SDLC-001	Version	1.0.0 — Final
Date	2026-06-11	Owner	Thapharan Vitayasiripaiboon (RD)
Asana task	Docs : Risking — [RD-M-26-07-Q1-KR6.4/AI] Sopify & Risk Management AI Strategy	Applies to	All projects under the sopify-sdlc skill

1. Purpose and Scope

This document defines how risk is identified, assessed, controlled, and escalated across the Sopify AI-assisted software development life cycle, in which a non-developer user builds and ships GS Battery internal web applications through Claude Cowork, GitHub, Supabase, and Railway.

It covers the full SOP-SDLC journey — Requirements, Design, Database, Development, Testing, Deploy, and the cross-cutting Git Workflow — and the multi-agent operating model (PM, UXUI Design, Frontend Engineer, Backend Engineer, Tester). It applies to every project run under the sopify-sdlc skill (SOP-SDLC, Code Standard SOP-DEV-001).

Out of scope: enterprise-level IT risk (network, endpoints, physical security), owned by GS Battery IT under separate policy.

2. Methodology — 5x5 Likelihood x Impact

Each risk is scored as Risk Score = Likelihood (L) x Impact (I), each rated 1-5.

Likelihood			Impact		
No.	Label	Meaning	No.	Label	Meaning
1	Rare	Hardly conceivable in a normal project run	1	Negligible	Cosmetic; no rework, no data or security effect
2	Unlikely	Could occur in exceptional circumstances	2	Minor	Small rework inside one phase
3	Possible	Could occur in some project runs	3	Moderate	Rework across phases, or schedule slip
4	Likely	Expected to occur in most runs without controls	4	Major	Production defect, outage, or significant rework
5	Almost certain	Will occur without controls	5	Severe	Data breach, data loss, or security incident

Risk level bands

Score	Level	Required response
15-25	High	Mitigation mandatory and verified at a gate; escalate on occurrence
8-12	Medium	Mitigation required; monitored at phase gates
1-6	Low	Accept with standard controls; review each cycle

The register is reviewed at every phase-gate decision and re-baselined at project kick-off and before each release (Release Gate 2).

3. Risk Register

14 identified risks. L = Likelihood, I = Impact, S = Score.

High risks (score 15-25)

ID	Risk	L	I	S	Owner	Mitigation
R-01	Secret leakage — Supabase service key, JWT secret, or .env committed to the repo or pasted into context/chat	4	5	20	IT + User (orchestrator)	Secrets only in env vars / Railway variables; .env never committed (.env.example only); tokens delivered by IT over a secure channel; reviewer greps for secrets at every PR; on exposure run the key-rotation procedure (Section 5) immediately.
R-02	Phase-gate skipping — downstream work starts on an unapproved upstream artifact (free non-linear navigation does not waive gates)	4	4	16	User (orchestrator)	Gate is the single completeness check before Push; Tester independently verifies every gate against files; docs/PHASE_STATE.md ledger re-read from disk before any delegation or gate decision; a worker's own pass is never sufficient.
R-03	Unapproved schema changes — SQL written or applied before the schema document is approved	3	5	15	Backend Engineer agent	Schema doc first, SQL second — Backend agent must stop for user approval via the orchestrator; approval recorded in the ledger; migrations only via Supabase MCP with UP/DOWN tested.
R-04	AI-hallucinated requirements — agent invents stories, AC, or scope the user never asked for; downstream phases build on them	4	4	16	PM agent + User	MoSCoW + user stories + AC produced by the PM agent from the user's raw brief only; user approves docs/requirements.md before design/database start; AC become the UAT scenarios so invented scope surfaces at Gate 2.
R-05	Untested AI-generated code merged — generated code passes superficial review but has no tests or hidden defects	4	4	16	Tester agent + Senior reviewer	Tests written alongside code (FE >= 70%, BE >= 80% coverage); Gate 1 on every PR (lint, type-check, tests, no debug code); Gate 2 before release (integration 100% of endpoints, k6 P95 < 2s, OWASP scan 0 critical); senior review with MUST/SHOULD/NIT labels.

Medium risks (score 8-12)

ID	Risk	L	I	S	Owner	Mitigation
R-06	Context drift ("lost in the middle") — SOP rules silently dropped in long sessions	3	3	9	User (orchestrator)	Multi-agent pattern: one phase = one fresh sub-agent receiving its SOP slice verbatim; artifacts passed as file paths, never pasted; ledger on disk is the source of truth.
R-08	Migration without tested rollback — schema change applied with no working DOWN path	3	4	12	Backend Engineer agent	Every migration has UP and commented DOWN; both tested against an empty DB before production; one logical change per migration; schema changelog maintained.
R-09	RLS missing on multi-tenant tables — data visible across tenants/users	2	5	10	Backend Engineer agent + IT	Phase 3 gate checks RLS policies; parameterized queries only; app DB user granted least privilege; go-live gate re-verifies RLS before deploy.
R-07	Direct push to protected branches — work bypasses PR, review, and CI	2	4	8	IT (repo admin)	Branch protection on main and develop (PR + passing CI + approval, no force-push); all work on type/asana-id-description branches; Railway deploys only from main, so nothing unreviewed reaches production.
R-10	Dependency vulnerabilities — known CVEs in npm packages reach production	3	3	9	Senior reviewer + IT	Dependencies checklist at project init; lockfiles committed; OWASP scan at Gate 2 with 0 critical findings; audit step in CI template.

ID	Risk	L	I	S	Owner	Mitigation
R-11	Traceability loss — work not linkable Asana - branch - PR - deploy	3	3	9	PM agent + User	Branch names carry the Asana task ID; every PR description links the Asana task (DoD item); Conventional Commits; ledger rows carry artifact paths and dates.

Low risks (score 1-6)

ID	Risk	L	I	S	Owner	Mitigation
R-12	Brand / accessibility non-compliance — UI drifts from the brand guideline or fails keyboard access	2	2	4	UXUI Design agent	Phase 2 gate: tokens match the guideline, 375px holds, Tab/Enter works, async states present; Figma Make prompts always state the brand guideline.
R-13	Deploy failure / downtime on Railway — bad build or config reaches production	2	3	6	IT + User	Pre-deploy checklist (local build green, tsc --noEmit, health check endpoint); smoke tests + post-deploy verification; Railway rollback to previous deploy if verification fails.
R-14	Knowledge concentration — a single non-dev operator holds all project context	2	3	6	PM agent + RD lead	All artifacts and state live in the repo (docs/, design/, migrations, docs/PHASE_STATE.md), not in chat history; any session can resume from the repo; SOP skill is versioned and self-contained.

4. Gate-Embedded Controls Mapping

Risk controls are not a separate audit layer — they are embedded in the SOP's existing quality gates. Skipping a gate therefore removes a control (see R-02).

SOP control point	Controls risks
Phase 1 gate — MoSCoW, AC per story, NFRs, user approval of requirements	R-04, R-11
Phase 2 gate — brand tokens, responsive, accessibility, async states	R-12
Phase 3 gate — schema doc approved before SQL; UP/DOWN tested; RLS; constraints	R-03, R-08, R-09
Phase 4 gate — Zod everywhere, no 'any', no secrets, standard response shape	R-01, R-05
Phase 6 Gate 1 (every PR) — lint, type-check, unit tests, coverage, no debug code	R-05
Phase 6 Gate 2 (pre-release) — integration 100%, k6 P95 < 2s, OWASP 0 critical, UAT sign-off	R-04, R-05, R-10, R-12
Phase 7 — branch protection, PR template + Asana link, senior review, Conventional Commits	R-01, R-02, R-07, R-11
Phase 5 go-live gate — secrets in Railway env, HTTPS, backups + RLS, smoke tests	R-01, R-09, R-13
docs/PHASE_STATE.md ledger + Tester gate verification	R-02, R-06, R-14

5. Escalation Path

1. Low — record in the register at the next gate review; standard controls apply.
2. Medium — phase owner pauses the affected work item, applies the mitigation, and the Tester re-verifies the gate before work resumes. PM informed at the next status update.
3. High — stop the affected phase immediately. Notify the PM and IT the same day. Work does not resume until the Tester re-verifies the relevant gate and the resolution is recorded in docs/PHASE_STATE.md and on the Asana task.

4. Secret exposure (R-01) — immediate, regardless of band: stop all pushes and deploys; IT rotates the exposed keys at once — regenerate Supabase anon/service keys, regenerate the production JWT secret (openssl rand -base64 32 — never reuse the dev value), update Railway environment variables; purge the secret from git history (rewrite or revoke — a deleted file is not a removed secret); verify the app on the rotated keys, run smoke tests, then resume; log the incident as an Asana task linked to the project with cause and resolution.

6. Review and Maintenance

This register is a living document: review it at every phase-gate decision, after any incident, and before each release. Material changes ship via the standard git workflow — a docs/ branch, a Conventional Commit, and an Asana-linked PR into develop — so the risk baseline is versioned with the code it protects.